

Argus: A Typed Interface Layer for Public-Coin Arguments and Reductions

Ricardo Perello

Final project report, June 2026

Abstract

Argus is a Rust interface layer for public-coin interactive arguments and reductions. It addresses a missing abstraction in the arkworks proof-system stack. Spongefish already provides the transcript and proof-string machinery needed for Fiat-Shamir. When a protocol calls that machinery directly, however, its message flow becomes mixed with sponge operations, serialization, and verifier replay. The resulting code describes a compiled transcript rather than an interactive protocol that can be reused independently of its execution backend.

Argus expresses the prover, verifier, and optional indexer algorithms as typed channel programs. Protocol code sends and receives messages; backends handle public-input binding, transcript ordering, challenge generation, proof bytes, and deterministic replay. The same interactive layer can run through a live backend, serve as input to the Duplex-Sponge Fiat-Shamir (DSFS) transformation, or become the output of a future compiler from a richer formalism such as an IOP with an iBCS commitment compiler. In the current architecture, each role is a native type with only the private data and dependencies it needs. This keeps the IA/IR layer as the central abstraction while making it suitable for separate deployment, preprocessing, composition, and future recursion.

1 Motivation

Many proof systems are naturally specified as public-coin interactive protocols. The prover sends algebraic values, commitments, or oracle-derived messages; the verifier samples public randomness; and the process continues until the verifier accepts or produces a reduced claim. A non-interactive implementation must additionally serialize messages, absorb them into a transcript, derive challenges, and replay the same sequence during verification.

In Spongefish’s Schnorr example, transcript calls appear directly inside proving and verification: the prover records messages and requests challenges, while the verifier reads proof bytes, absorbs them, derives the same challenges, and checks end-of-file [2]. This is the right abstraction for a transcript library. It is not the right abstraction for describing an interactive argument as a reusable protocol. The protocol and its compiled transcript are entangled in one imperative program.

Argus inserts the missing protocol layer. A role describes only the protocol’s message flow: which prover messages are sent, which public verifier messages are returned, and what the verifier outputs. An execution backend supplies the transport or transcript. Consequently, an IA or IR is not defined by DSFS. It can run interactively, be compiled into a non-interactive argument, or be generated by another compiler.

This separation also fits into `arkworks-pq`, a broader effort to develop post-quantum proof-system components for arkworks. Its roadmap includes vector commitments, Merkle trees, FRI,

STIR-, WHIR-, and STARK-related components, sumcheck, linear codes, and binary-field primitives. The intended pipeline is not only

$$IA \xrightarrow{DSFS} NARG,$$

but eventually

$$IOP \xrightarrow{iBCS} IA \xrightarrow{DSFS} NARG.$$

The IA layer is therefore a backend input, a target for higher-level compilation, and an independently executable protocol interface.

2 Design Objectives

Argus covers three semantic axes:

Axis	Cases
Verifier output	argument / reduction
Execution mode	interactive / non-interactive
Setup model	plain / preprocessed

An argument ends in accept or reject. A reduction instead outputs an instance of a target relation. Non-interactive forms are produced by a backend such as DSFS. Preprocessed forms additionally use an index, a prover key, a verifier key, and a committed representation of the indexed material.

The interface must represent these forms without giving every type every operation. It must also avoid duplicating transcript logic across them, keep Rust’s type checker useful, permit realistic protocol configuration, and preserve exact proof-format compatibility. Finally, the abstraction must model real deployment boundaries: proving, verification, and indexing may occur in different crates, binaries, or machines.

The three axes classify the protocol itself. Separately, each executable form has a prover role and a verifier role, and preprocessing adds an indexer. Together, compatible roles realize an IA or IR, but no single type must contain the whole protocol. An earlier Argus design placed the prover and verifier methods on one protocol type. That made small experiments concise, but it also made verifier code depend on prover-only types and implementations. The current interface keeps the same channel abstraction while representing the roles as independent types.

3 Typed Channel Programs

The channel traits form the boundary between protocol logic and execution machinery. A prover role may only send prover messages and read verifier messages. A verifier role may only read prover messages and send verifier messages. Protocol code has no sponge object, absorb call, Fiat–Shamir derivation, or proof-byte parser.

$$\begin{array}{ccccccc} P & \longrightarrow & C_P & \xrightarrow{\text{prover message}} & C_V & \longrightarrow & V \\ P & \longleftarrow & C_P & \xleftarrow{\text{verifier message}} & C_V & \longleftarrow & V \end{array}$$

Here P and V are the protocol roles, while C_P and C_V are the prover- and verifier-side channel implementations supplied by a backend. In live execution the two channels relay messages, and the verifier samples the public coins directly. The `spongefsh-dsfs` crate implements the same

interfaces without a live connection: prover messages become proof bytes and are absorbed into the sponge, while verifier messages are derived as challenges; verification reads the proof and repeats the same derivation. A future iBCS compiler can target the same role interfaces after translating oracle commitments and openings into an interactive argument.

This design centralizes the transcript invariants required by DSFS Construction 4.3 [1]. Protocol identity, session data, committed index where applicable, and the instance are bound before the first challenge. Every prover message is absorbed before the following challenge is squeezed. Verification is deterministic. After the protocol finishes, the verifier checks that no proof bytes remain unread; the test suite explicitly rejects proofs with an appended byte. Sponge operations occur only in the backend.

Channels do not prove that a protocol is secure. They impose an implementation discipline: protocol authors specify the message flow, while one reviewed backend controls transcript ordering and replay. Security still depends on the protocol analysis, the DSFS theorem, correct serialization, and the backend implementation.

4 Role-First Architecture

Argus represents the prover and verifier as separate concrete types. They implement the same public protocol format, but each type exposes only the data and methods required for its role. For arguments, **ArgumentCore** contains only the public instance type, while **ArgumentProverCore** adds the witness. Reduction cores similarly separate source and target instances from their witnesses. A verifier therefore does not import prover-private witness types.

Executable traits are role-specific. For example, interactive arguments have separate prover and verifier traits, with corresponding reduction, non-interactive, and preprocessing forms. There is no required all-in-one trait that combines both roles. Authors implement separate concrete prover and verifier types. When both roles use the same generic bounds, one macro declaration can generate the two implementations. When their bounds differ or the roles live in different crates, each implementation can be declared separately. The two types may share a public format descriptor while carrying different local configuration and dependencies.

DSFS also compiles roles explicitly:

$$\begin{aligned} IA_P &\xrightarrow{DSFS_P} NIA_P, \\ IA_V &\xrightarrow{DSFS_V} NIA_V. \end{aligned}$$

These arrows correspond to distinct DSFS output types. The compiled prover contains the prover role and exposes **prove**; the compiled verifier contains the verifier role and exposes **verify**. DSFS does not combine them into another protocol object. An application that needs both keeps the two compiled values side by side. This is important for recursion and accumulation, where an outer prover may need an inner verifier without depending on the inner prover.

Independent roles must agree on the public protocol contract: **protocol_id**, instance and message encodings, sponge and salt policy, session encoding, committed-index encoding, and transcript layout. Associated types enforce much of the public shape at integration points; protocol identifiers and compatibility tests bind the remaining format choices. Matching this contract does not replace a correctness proof, but it makes deployment compatibility explicit rather than hiding it inside one concrete type.

5 Configuration, Indexing, and Domain Separation

Real protocols carry configuration that is neither a per-claim instance nor a witness. WHIR- and FRI-style systems may contain rates, folding schedules, query counts, code information, and hash choices. Argus permits each role to store immutable configuration in `&self`. Prover-local performance settings, verifier resource limits, and indexer storage settings may differ.

Any configuration that changes the accepted language, message flow, challenges, proof layout, soundness profile, or transcript interpretation must be reflected in `protocol_id(&self)` or otherwise bound before the first challenge. Configuration that changes only local performance need not change the identifier. Transcript state itself is never role configuration: mutable sponge state and challenge counters belong to the backend.

For preprocessed protocols, indexing is a separate role:

$$i \mapsto (pk_i, vk_i).$$

Only the **Indexer** knows the index and both key types. The prover receives a prover key, while the verifier receives a verifier key and has no witness type in its interface. Keys remain explicit inputs rather than hidden state inside prepared protocol objects, so one compiled role can be reused across many indexes.

Both keys expose canonical committed-index bytes. The **Indexer** contract requires keys derived from the same index to expose identical bytes; Argus does not perform a runtime check. A violation makes prover and verifier initialize different transcripts, so verification fails. During execution, DSFS binds the relevant key commitment together with the ordinary instance before the first challenge. DSFS compiles executable roles only; it neither stores nor forwards the indexer.

6 Composition

Reductions are first-class because many proof systems transform a claim rather than immediately deciding it:

$$R_0 \xrightarrow{IR_1} R_1 \xrightarrow{IR_2} R_2.$$

Binary chaining gives arbitrary finite sequential compositions, and a reduction followed by an argument becomes an argument for the original relation.

Composition is role-specific. Prover composition relates target instances and target witnesses and invokes only prover components. Verifier composition relates only public target instances and invokes only verifier components; it has no witness equality bounds. Indexers compose separately, pairing their indexes, prover keys, verifier keys, and committed-index encodings. Thus the three composition paths are

$$\begin{aligned} \text{prover components} &\rightarrow \text{composed prover,} \\ \text{verifier components} &\rightarrow \text{composed verifier,} \\ \text{indexers} &\rightarrow \text{composed indexer.} \end{aligned}$$

This introduces some forwarding boilerplate, but preserves the dependency boundary through composed protocols. The current model is deliberately sequential rather than a general protocol-DAG language.

7 Results

Argus implements the IA/IR trait layer, a live backend, the **spongefish-dsfs** compiler backend, composition adapters, security metadata, examples, and larger case studies. Schnorr and sumcheck

demonstrate the basic channel model. Bulletproofs exercises both a direct logarithmic inner-product argument and the same computation expressed as a one-round reduction composed to a size-one decider [4].

WARP is the largest case study: a linear-time accumulation scheme for R1CS represented as preprocessing reductions followed by a final preprocessing argument [3]. Its configuration, code information, Merkle parameters, and relation data are indexed, while fresh instances and accumulators remain per-claim inputs. The port also exposes the principal missing abstraction: oracle commitments and openings are currently encoded ad hoc as IA/IR messages. A general IOP-to-IA compiler should generate this layer instead.

The role-first migration covers argument and reduction roles in plain and preprocessed settings, with independent DSFS constructors and live execution. Tests check role-specific composition, indexing failures, deterministic replay, EOF rejection, and transcript compatibility. Existing proof fixtures remain byte-identical, including the sigma-proofs SHAKE128 vectors. The migration therefore changed the type structure and dependencies without changing the underlying DSFS transcript.

The sigma bridge is a smaller compatibility experiment. It rewrites the non-interactive pipeline used by `sigma-proofs` in the Argus style. Because the existing upstream path is already used by other consumers, the bridge serves as interoperability evidence rather than a proposed replacement.

8 Limitations and Future Work

Argus is a research prototype and has not received an independent cryptographic review. Typed channels reduce transcript mistakes but do not establish protocol security.

The most important missing component is the IOP-to-IA compiler. Implementing the iBCS step would test Argus as a target interface and replace WARP’s ad hoc oracle-opening messages with a reusable compiler layer. A recursive or accumulation consumer should also validate the practical benefit of depending on a native verifier without the corresponding prover. Separate role types in one crate provide type-level isolation but not crate-level dependency isolation; demanding deployments may need separate format, prover, verifier, and indexer crates.

The codec boundary remains close to Spongefish’s serialization vocabulary. It is open whether `ia-core` should define smaller encoding traits and how field-element transcript alphabets should fit beside the current byte-oriented DSFS proof machinery. Deeply nested composition types and preprocessing tuples also remain ergonomically heavy.

9 Conclusion

Argus contributes the interactive protocol layer that was missing between proof-system descriptions and transcript backends in the arkworks stack. It expresses public-coin arguments and reductions as typed channel programs, keeping transcript mechanics in DSFS or a live execution backend. The same layer can be consumed by DSFS, executed interactively, or targeted by a future iBCS compiler.

The current architecture expresses the prover, verifier, and indexer as independent roles that agree on the public protocol format. This keeps the IA/IR abstraction clean while supporting realistic dependencies, configuration, preprocessing, deployment, composition, and future recursion.

References

- [1] Alessandro Chiesa and Michele Orru. *A Fiat-Shamir Transformation From Duplex Sponges*. IACR ePrint 2025/536, 2025. <https://eprint.iacr.org/2025/536>
- [2] arkworks-rs spongefish. Schnorr example. <https://github.com/arkworks-rs/spongefish/blob/main/spongefish/examples/schnorr.rs>
- [3] Benedikt Bünz, Alessandro Chiesa, Giacomo Fenzi, and William Wang. *Linear-Time Accumulation Schemes*. IACR ePrint 2025/753, 2025. <https://eprint.iacr.org/2025/753>
- [4] Benedikt Bünz et al. *Bulletproofs*. IEEE S&P, 2018.